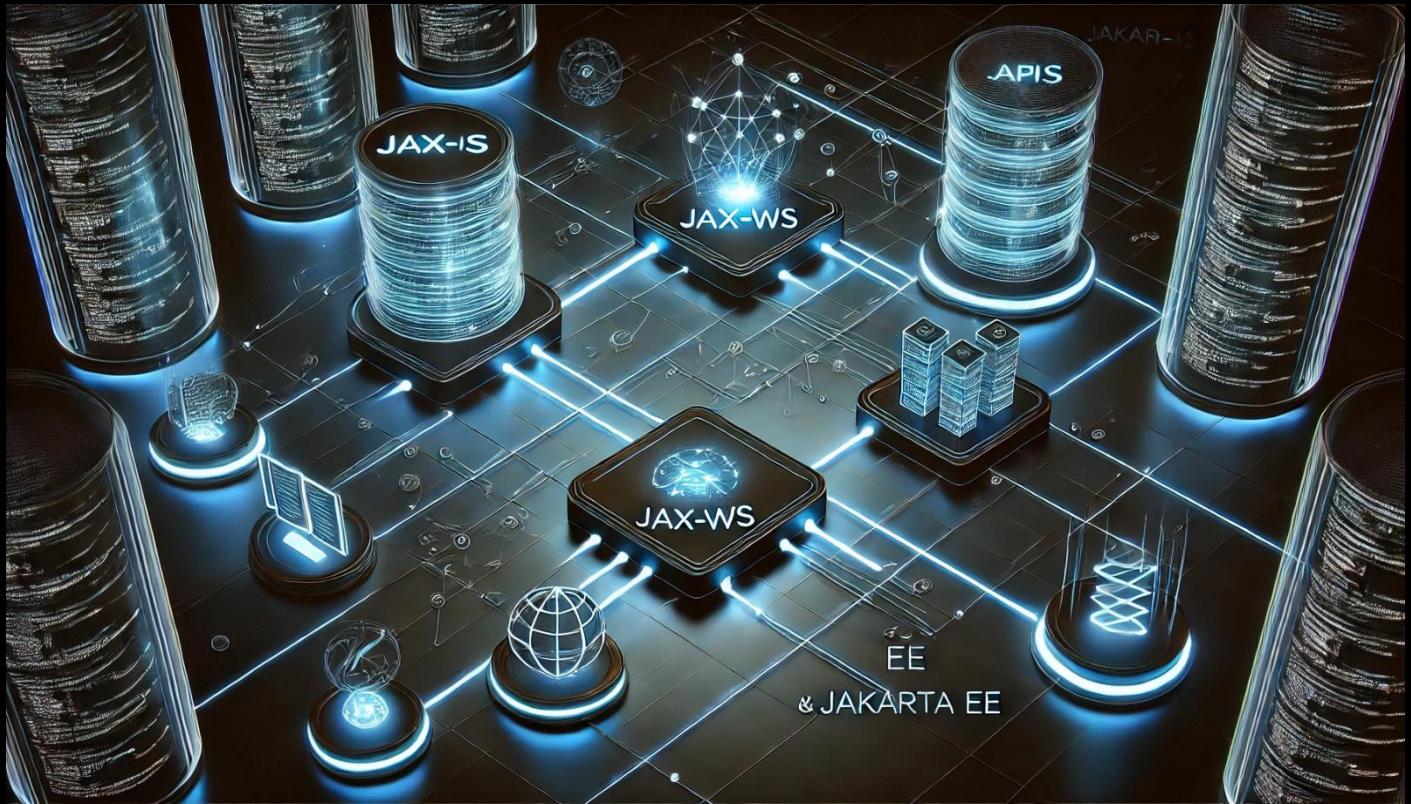


SGA con Web Service JAX-WS y Jakarta EE



Guía: Creación del Web Service listado de Personas con JAX-WS

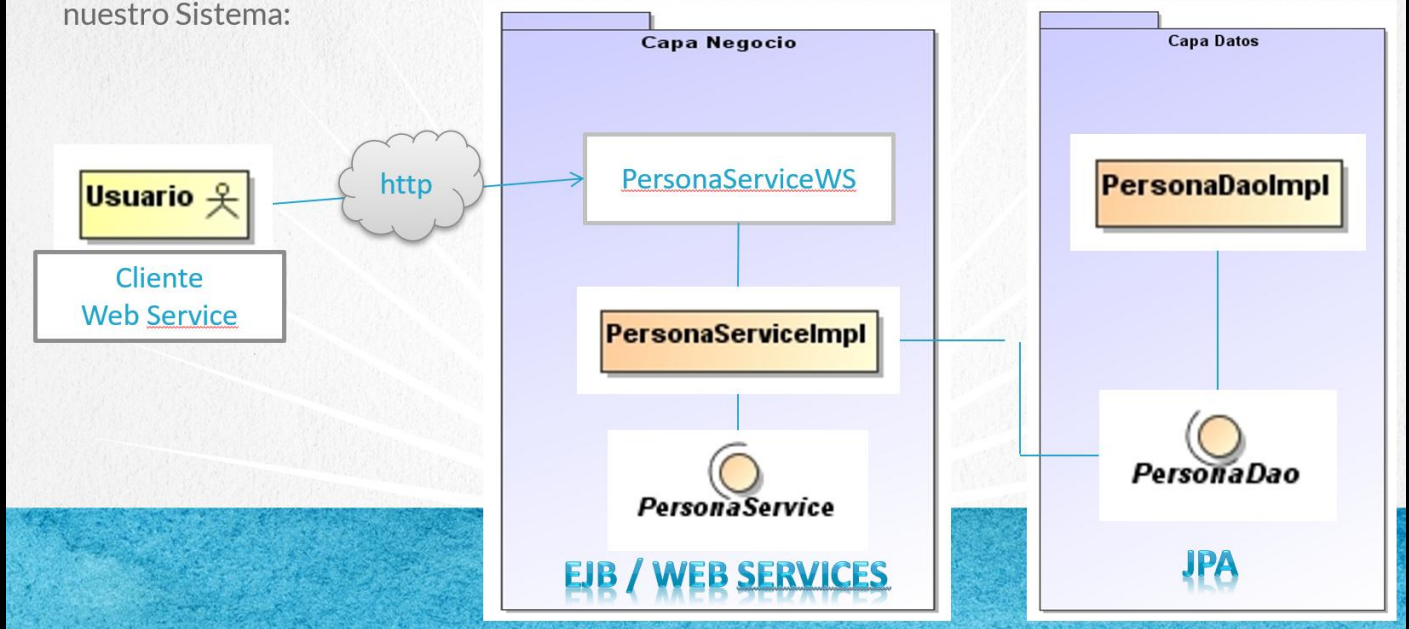
Introducción

En esta lección vamos a **exponer un servicio web** en nuestro proyecto `sga-jee-web` utilizando la tecnología **JAX-WS** (Java API for XML Web Services) . El objetivo es permitir que **otros sistemas o clientes** puedan consumir la funcionalidad de nuestro sistema, en este caso, el **listado de personas**.

A continuamos mostramos la arquitectura que vamos a utilizar para crear el servicio web:

Arquitectura sga con web services

Este es el Diagrama de Clases del Ejercicio, donde se pueden observar la Arquitectura de nuestro Sistema:



🧩 Paso 1: Abrir el proyecto

Desde NetBeans, abrimos el proyecto si aún no lo tenemos abierto:

1. Seguimos trabajando con el proyecto `sga-jee-web`.

🛑 Paso 2: Detener GlassFish

Antes de desplegar o volver a compilar, detén cualquier instancia en ejecución:

1. Haz clic derecho sobre el servidor Services -> GlassFish Server
2. Dar clic derecho y Selecciona **Stop**.

🔨 Paso 3: Cerrar el proyecto anterior

En la pestaña `Projects`:

- Haz clic derecho sobre los proyectos que ya no estén usando.

- Selecciona **Close**.

🔧 Paso 4: Agregar dependencias para Web Services

Abre el archivo `pom.xml` ubicado en:

`sga-jee-web/pom.xml`

Agrega la siguiente dependencia para incluir el API de JAX-WS:

```
<dependency>
  <groupId>jakarta.jws</groupId>
  <artifactId>jakarta.jws-api</artifactId>
  <version>3.0.0</version>
</dependency>
```

✂ Hacer clean & Build para que descargue la dependencia. Esta dependencia es esencial para usar las anotaciones como `@WebService`.

Código `pom.xml` completo:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>sga</groupId>
  <artifactId>sga-jee-web</artifactId>
  <version>1.0</version>
  <packaging>war</packaging>
  <name>sga-jee-web</name>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <jakartaee>11.0.0-M1</jakartaee>
  </properties>

  <dependencies>
    <dependency>
      <groupId>jakarta.platform</groupId>
      <artifactId>jakarta.jakartaee-api</artifactId>
      <version>${jakartaee}</version>
      <scope>provided</scope>
    </dependency>
```

```
<dependency>
  <groupId>jakarta.persistence</groupId>
  <artifactId>jakarta.persistence-api</artifactId>
  <version>3.2.0</version>
</dependency>
<dependency>
  <groupId>org.eclipse.persistence</groupId>
  <artifactId>eclipselink</artifactId>
  <version>4.0.5</version>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>9.2.0</version>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>2.0.16</version>
  <type>jar</type>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-simple</artifactId>
  <version>2.0.16</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.primefaces</groupId>
  <artifactId>primefaces</artifactId>
  <version>15.0.0</version>
  <classifier>jakarta</classifier>
</dependency>
<dependency>
  <groupId>jakarta.jws</groupId>
  <artifactId>jakarta.jws-api</artifactId>
  <version>3.0.0</version>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
```

```

        <version>3.12.1</version>
        <configuration>
            <source>21</source>
            <target>21</target>
        </configuration>
    </plugin>
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
    </plugin>
</plugins>
</build>
</project>

```



Paso 5: Crear la interfaz del Web Service

Ubicación del archivo:

```

src/main/java/sga/servicio/PersonaServiceWs.java
package sga.servicio;

```

```

import java.util.List;
import jakarta.jws.WebMethod;
import jakarta.jws.WebService;
import sga.domain.Persona;

@WebService
public interface PersonaServiceWs {

    @WebMethod
    List<Persona> listarPersonas();
}

```



Esta interfaz define el contrato que será expuesto como Web Service.



Paso 6: Modificar la implementación del servicio

Ubicación del archivo:

```

src/main/java/sga/servicio/PersonaServiceImpl.java

```

Agrega las anotaciones correspondientes:

```
@Stateless
@WebService(endpointInterface = "sga.servicio.PersonaServiceWs")
public class PersonaServiceImpl implements PersonaServiceRemote, PersonaService, PersonaServiceWs{
    // ya se encuentra implementado el método listarPersonas y demás métodos
}
```

 Con esto, indicamos que esta clase implementa la interfaz expuesta como servicio web.

Código `PersonaServiceImpl.java` completo:

```
package sga.servicio;

import java.util.List;
import jakarta.ejb.Stateless;
import jakarta.inject.Inject;
import jakarta.jws.WebService;
import sga.datos.PersonaDao;
import sga.domain.Persona;

@Stateless
@WebService(endpointInterface = "sga.servicio.PersonaServiceWs")
public class PersonaServiceImpl implements PersonaServiceRemote, PersonaService, PersonaServiceWs{

    @Inject
    private PersonaDao personaDao;

    @Override
    public List<Persona> listarPersonas() {
        return personaDao.findAllPersonas();
    }

    @Override
    public Persona encontrarPersonaPorId(Persona persona) {
        return personaDao.findPersonaById(persona);
    }

    @Override
    public Persona encontrarPersonaPorEmail(Persona persona) {
        return personaDao.findPersonaByEmail(persona);
    }

    @Override
    public void registrarPersona(Persona persona) {
```



```
        personaDao.insertPersona(persona);
    }

    @Override
    public void modificarPersona(Persona persona) {
        personaDao.updatePersona(persona);
    }

    @Override
    public void eliminarPersona(Persona persona) {
        personaDao.deletePersona(persona);
    }
}
```



Paso 7: Preparar la clase `Persona` para convertirla a XML

Ubicación del archivo:

```
src/main/java/sga/domain/Persona.java
```

Agrega las siguientes anotaciones al inicio de la clase:

```
import jakarta.xml.bind.annotation.XmlAccessorType;
import jakarta.xml.bind.annotation.XmlAccessType;
import jakarta.xml.bind.annotation.XmlTransient;

@XmlAccessorType(XmlAccessType.FIELD)
public class Persona {
    ...

    @XmlTransient
    private List<Usuario> usuarios;
}
```

📦 Esto permitirá que los atributos de la clase `Persona` se conviertan a XML correctamente, y omitimos la lista de usuarios para evitar problemas de rendimiento o llamadas circulares.

Código `Persona.java` completo:

```
package sga.domain;

import jakarta.persistence.*;
```

```

import jakarta.validation.constraints.Size;
import jakarta.xml.bind.annotation.XmlAccessType;
import jakarta.xml.bind.annotation.XmlAccessorType;
import jakarta.xml.bind.annotation.XmlTransient;
import java.io.Serializable;
import java.util.List;

@Entity
@NamedQueries({
    @NamedQuery(name = "Persona.findAll", query = "SELECT p FROM Persona p"),
    @NamedQuery(name = "Persona.findByIdPersona", query = "SELECT p FROM Persona p WHERE p.idPersona = :idPersona"),
    @NamedQuery(name = "Persona.findByIdNombre", query = "SELECT p FROM Persona p WHERE p.nombre = :nombre"),
    @NamedQuery(name = "Persona.findByIdApellido", query = "SELECT p FROM Persona p WHERE p.apellido = :apellido"),
    @NamedQuery(name = "Persona.findByIdEmail", query = "SELECT p FROM Persona p WHERE p.email = :email"),
    @NamedQuery(name = "Persona.findByIdTelefono", query = "SELECT p FROM Persona p WHERE p.telefono = :telefono")})
@XmlAccessorType(XmlAccessType.FIELD)
public class Persona implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "id_persona")
    private Integer idPersona;
    @Size(max = 45)
    private String nombre;
    @Size(max = 45)
    private String apellido;
    // @Pattern(regexp="[a-z0-9!#$%&'*/+=?^_`{|}~-]+(?:\\. [a-z0-9!#$%&'*/+=?^_`{|}~-]+)*@(?:[a-z0-9](?:[a-z0-9]*[a-z0-9])?\\.)+[a-z0-9](?:[a-z0-9]*[a-z0-9])?", message="Invalid email")//if the field
    // contains email address consider using this annotation to enforce field validation
    @Size(max = 45)
    private String email;
    @Size(max = 45)
    private String telefono;

    @XmlTransient
    @OneToMany(mappedBy = "persona", cascade = CascadeType.ALL)
    private List<Usuario> usuarioList;

    public Persona() {

```



```
}

public Persona(String nombre, String apellido, String email, String telefono) {
    this.nombre = nombre;
    this.apellido = apellido;
    this.email = email;
    this.telefono = telefono;
}

public Persona(Integer idPersona) {
    this.idPersona = idPersona;
}

public Integer getIdPersona() {
    return idPersona;
}

public void setIdPersona(Integer idPersona) {
    this.idPersona = idPersona;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}
```

```
public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
    this.telefono = telefono;
}

public List<Usuario> getUsuarioList() {
    return usuarioList;
}

public void setUsuarioList(List<Usuario> usuarioList) {
    this.usuarioList = usuarioList;
}

@Override
public int hashCode() {
    int hash = 0;
    hash += (idPersona != null ? idPersona.hashCode() : 0);
    return hash;
}

@Override
public boolean equals(Object object) {
    // TODO: Warning - this method won't work in the case the id fields are not set
    if (!(object instanceof Persona)) {
        return false;
    }
    Persona other = (Persona) object;
    if ((this.idPersona == null && other.idPersona != null) || (this.idPersona != null &&
!this.idPersona.equals(other.idPersona))) {
        return false;
    }
    return true;
}

@Override
public String toString() {
    return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", apellido=" + apellido +
", email=" + email + ", telefono=" + telefono + '}';
}
}
```



Paso 8: Ejecutar y desplegar el proyecto

1. Haz clic derecho en el proyecto → **Clean and Build**.
2. Asegúrate de que GlassFish esté detenido y luego inícialo.
3. Haz clic derecho en el proyecto → **Run**.

Verifica que el WAR se ha generado y desplegado correctamente.



Paso 9: Probar el Web Service

Abre la consola de administración de GlassFish:

- En Services, da Click derecho sobre GlassFish → **View Domain Admin Console**.
- Ve a **Applications** → selecciona `sga-jee-web`. y confirmamos que además de EJB, ya incluye Web Services.



También puedes acceder directamente al WSDL en:

<http://localhost:8080/PersonaServiceImplService/PersonaServiceImpl?wsdl>

Y para probar el servicio desde el navegador:

<http://localhost:8080/PersonaServiceImplService/PersonaServiceImpl?Tester>

Haz clic en **listarPersonas** y verás los datos convertidos a XML.

Method parameter(s)

Type	Value

Method returned

```
java.util.List : "[sga.servicio.Persona@6f76d6c5, sga.servicio.Persona@56530f57, sga.servicio.Persona@5d1cd291, sga.servicio.Persona@76025d9]"
```

SOAP Request

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <S:Body>
    <ns2:listarPersonas xmlns:ns2="http://servicio.sga/"/>
  </S:Body>
</S:Envelope>
```

SOAP Response

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <S:Body>
    <ns2:listarPersonasResponse xmlns:ns2="http://servicio.sga/">
      <return>
        <idPersona>1</idPersona>
        <nombre>Rafael</nombre>
        <apellido>Juarez</apellido>
        <email>rjuarez@mail.com</email>
        <telefono>55667788</telefono>
      </return>
      <return>
        <idPersona>2</idPersona>
        <nombre>Ivonne</nombre>
        <apellido>Lara</apellido>
        <email>ilara@mail.com</email>
        <telefono>55443322</telefono>
      </return>
    </ns2:listarPersonasResponse>
  </S:Body>
</S:Envelope>
```



Conclusión

En este ejercicio:

- ✓ Expusimos el EJB `PersonaService` como un Web Service.
- ✓ Creamos la interfaz con anotaciones JAX-WS.
- ✓ Preparamos la clase `Persona` para su conversión a XML.
- ✓ Probamos el servicio desde el navegador.

🥳 ¡Todo un logro! Ya estás listo para consumir este servicio desde un cliente en la siguiente lección.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)